INFRASTRUCTURE AUTOMATION DONE RIGHT

# TERRAGRUNT PRODUCTION HANDBOOK

## From Terraform Reuse to Production Infrastructure at Scale


- DRY CONFIGURATION
- ENVIRONMENT MANAGEMENT
- DEPENDENCY AWARENESS
- REMOTE STATE MANAGEMENT
- INFRASTRUCTURE AS CODE AT SCALE
- SAFE, REPEATABLE, AND AUDITABLE DEPLOYMENTS

terragrunt.hcl

```
terrafarm {
  source = "git::ssh://...//vpc"
}
inputs = {
  environment = "prod"
  region      = "us-east-1"
}
```

**BUILD SMARTER. DEPLOY SAFER. SCALE WITH CONFIDENCE.**

# TERRAGRUNT EXPLAINED,

## WHY TERRAFORM TEAMS USE IT

### 1. WHAT TERRAGRUNT IS

Terragrunt is a thin wrapper for Terraform that adds extra tools for working with multiple environments, accounts, and reusable configurations at scale.

### 2. THE INFRASTRUCTURE PROBLEM IT SOLVES

As infrastructure grows, teams face:

- Repeated Terraform configuration
- Multiple environments (dev, staging, prod)
- Multiple accounts and regions
- Hard to keep settings consistent
- Copy-paste everywhere
- High risk of drift and human error
- Difficult to scale the team safely

### 3. COMMON PAIN: REPEATED TERRAFORM CONFIGURATION

```
provider "aws" {
  region = "us-east-1"
}
terraform {
  backend "s3" {
  bucket = "my-tf-state"
  key    = "dev/terraform.tfstate"
  region = "us-east-1"
 }
}
```

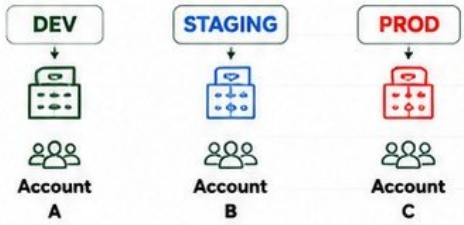
→ DEV (duplicate)

→ STAGING (duplicate)

→ PROD (duplicate)

The same settings are copied and modified in every environment. This is hard to maintain and error-prone.

### 4. MULTIPLE ENVIRONMENTS & ACCOUNTS




Terragrunt helps you manage all of this without duplication and confusion.

### 5. KEEPING TERRAFORM CODE DRY

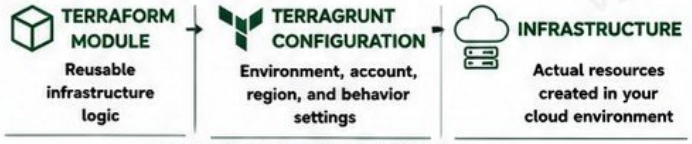
Terragrunt lets you centralize common settings and reuse Terraform modules across many environments.

- One place for backend config
- One place for provider config
- One set of modules
- Many environments
- Many accounts
- Consistent and safer deployments

### 6. TERRAFORM vs TERRAGRUNT

| TERRAFORM | TERRAGRUNT |
|---|---|
| Infrastructure as Code engine | Thin wrapper and orchestration layer |
| Works with one configuration at a time | Manages many configs across environments |
| Requires repeated backend/provider setup | Makes it DRY and consistent |
| No built-in awareness of dependencies | Handles dependencies between stacks |
| Manual orchestration | Smarter orchestration and automation |

### 7. MENTAL MODEL




MODULE IS THE BLUEPRINT.
TERRAGRUNT IS HOW AND WHERE YOU USE IT.

### 8. JUNIOR ENGINEER TIP

Terragrunt does not replace Terraform knowledge.
You still need to understand modules, resources, state, providers, and core Terraform concepts.
Terragrunt simply helps you use Terraform better at scale.

# TERRAFORM BEFORE TERRAGRUNT,
## UNDERSTANDING THE PROBLEM

### ⚠ THE PROBLEM WITH TERRAFORM ALONE

**REPEATED BACKEND CONFIGURATION**
Every environment has its own backend block copied everywhere.

**REPEATED PROVIDER CONFIGURATION**
Providers, versions, and settings are defined in every environment.

**DEV, STAGING, AND PRODUCTION DUPLICATION**
The same code structure is duplicated for each environment.

**COPY-AND-PASTE INFRASTRUCTURE**
Changes must be made in many places, increasing human error.

**CONFIGURATION DRIFT**
Small differences appear between environments over time.

**GROWING REPOSITORY COMPLEXITY**
As environments, regions, and accounts grow, the repo becomes hard to manage.

**WHY DUPLICATION BECOMES DANGEROUS**
- Inconsistent infrastructure
- Harder to maintain and review
- Higher risk of outages
- Slower delivery
- Harder onboarding for new engineers

### THE PROBLEM: DUPLICATED TERRAFORM CONFIGURATION

**DEV**
- main.tf
- variables.tf
- backend.tf
- provider.tf
- outputs.tf

COPIED SETTINGS
Backend
...
Provider
...

**STAGING**
- main.tf
- variables.tf
- backend.tf
- provider.tf
- outputs.tf

COPIED SETTINGS
Backend
...
Provider
...

**PROD**
- main.tf
- variables.tf
- backend.tf
- provider.tf
- outputs.tf

COPIED SETTINGS
Backend
...
Provider
...

### THE SOLUTION: CENTRALIZED WITH TERRAGRUNT

**TERRAGRUNT ROOT (CENTRALIZED)**
- backend.hcl
- providers.hcl
- common.hcl
- versions.hcl
- locals.hcl

SHARED SETTINGS
Backend
...
Provider
...

**DEV**
terragrunt.hcl

INPUTS
region = "us-east-1"
env = "dev"
...

**STAGING**
terragrunt.hcl

INPUTS
region = "us-east-1"
env = "staging"
...

**PROD**
terragrunt.hcl

INPUTS
region = "us-east-1"
env = "prod"
...

✓ ONE PLACE FOR COMMON SETTINGS.
CLEAN, CONSISTENT, AND EASY TO SCALE.

### JUNIOR ENGINEER TIP
Terragrunt brings structure, reuse, and safety to Terraform.
It helps your team manage infrastructure at scale without the chaos.

# INSTALLING TERRAGRUNT
## AND YOUR FIRST COMMANDS

### 1. PREREQUISITES

- A working system (Linux, macOS, or Windows)
- Terraform or OpenTofu must be installed
- Access to the cloud provider
- Basic understanding of Terraform
- Git installed (recommended)

TERRAGRUNT REQUIRES TERRAFORM OR OPENTOFU TO WORK.

TERRAGRUNT DOES NOT DEPLOY INFRASTRUCTURE BY ITSELF.

### 2. INSTALLING TERRAGRUNT

**LINUX (USING CURL)**

```
curl -L https://terragrunt.gruntwork.io/downloads/latest/linux_amd64 -o terragrunt
chmod +x terragrunt && sudo mv terragrunt /usr/local/bin/
```

**macOS (USING HOMEBREW)**

```
brew install terragrunt
```

**WINDOWS (USING CHOCOLATEY)**

```
choco install terragrunt
```

**MANUAL DOWNLOAD**

Visit: https://terragrunt.gruntwork.io/docs/getting-started/install/
Download the appropriate binary for your system.

### 3. YOUR FIRST COMMANDS

| | | |
|---|---|---|
| Check Terragrunt version<br>Verify installation. | `terragrunt --version` | Shows Terragrunt version. |
| Initialize the working directory<br>Downloads modules and prepares the backend. | `terragrunt init` | Prepares everything needed for execution. |
| Generate an execution plan<br>Shows what changes will be made. | `terragrunt plan` | Runs terraform plan under the hood. |
| Apply the changes<br>Deploy infrastructure. | `terragrunt apply` | Runs terraform apply under the hood. |
| Destroy the infrastructure<br>Tear down everything managed. | `terragrunt destroy` | Runs terraform destroy under the hood. |

### 4. HOW TERRAGRUNT WORKS

**YOU RUN TERRAGRUNT COMMAND**
e.g. `terragrunt plan`

↓

**TERRAGRUNT READS terragrunt.hcl**
Loads configuration, inputs, remote state, dependencies, etc.

↓

**TERRAGRUNT GENERATES CONTEXT**
Creates backend, provider, and other necessary files.

↓

**TERRAGRUNT CALLS TERRAFORM**
Executes the appropriate Terraform command.

↓

**INFRASTRUCTURE MANAGED**
Resources are created, updated, or destroyed in the cloud.

### 5. PRACTICE LAB

1. Create a new folder for your demo
2. Write a simple Terraform module
3. Create terragrunt.hcl pointing to the module
4. Run terragrunt init
5. Run terragrunt plan and review the output
6. Run terragrunt apply to create resources
7. Verify resources in your cloud account
8. Finally, run terragrunt destroy

### JUNIOR ENGINEER TIP

Terragrunt does not replace Terraform.
You still need to understand Terraform modules, providers, state files, variables, and cloud infrastructure.
Terragrunt simply makes your Terraform workflow scalable, DRY, and safer.

# UNDERSTANDING terragrunt.hcl

## THE HEART OF TERRAGRUNT CONFIGURATION

### 1. PURPOSE OF TERRAGRUNT.HCL

**terragrunt.hcl** is the central configuration file that tells Terragrunt:

- Which Terraform module to use
- How to configure it
- Where and how to store state
- What other stacks it depends on
- How to generate the context for Terraform

**IT KEEPS INFRASTRUCTURE DRY, CONSISTENT, AND SCALABLE.**

### 2. HCL BASICS (HASHICORP CONFIGURATION LANGUAGE)

- HCL uses blocks, attributes, and values.
- Blocks contain settings.
- Attributes use key = value.
- Supports strings, numbers, lists, maps, and expressions.
- Example:

```
inputs = {
  name  = "web"
  instance_type = "t3.medium"
  tags = {
    env = "dev"
  }
}
```

### 3. KEY BLOCKS IN TERRAGRUNT.HCL

**terraform BLOCK**
Defines the Terraform settings.
Typically used to specify required version.

**source ATTRIBUTE**
Points to the Terraform module source.
Can be local path or remote (Git).

**inputs BLOCK**
Passes variables to the Terraform module.
This is how you customize your infrastructure.

**locals BLOCK**
Defines local values to reuse within the configuration.

**include BLOCK**
Inherits configuration from parent terragrunt.hcl files to avoid duplication.

**dependency BLOCK**
Reads outputs from other Terragrunt stacks that this stack depends on.

**remote_state BLOCK**
Configures where and how Terraform state is stored.

**TIP**
These blocks work together to build the execution context for Terraform.

### 4. HOW TERRAGRUNT GENERATES TERRAFORM EXECUTION CONTEXT

| | |
|---|---|
| READ terragrunt.hcl | Terragrunt reads the configuration and all included files. |
| ↓ | |
| MERGE CONFIGURATIONS | Includes, locals, and inputs are merged into final values. |
| ↓ | |
| GENERATE BACKEND & PROVIDER CONFIG | Terragrunt generates necessary Terraform files (backend.tf, provider.tf). |
| ↓ | |
| EXECUTE TERRAFORM | Terragrunt runs Terraform commands with the generated context. |
| ↓ | |
| MANAGE INFRASTRUCTURE | Resources are created, updated, or destroyed in your cloud. |

### 5. ANATOMY OF A PRODUCTION TERRAGRUNT.HCL

```
# terragrunt.hcl (example)
terraform {
  source = "git::ssh://git@github.com/org/vpc.git//modules/vpc?ref=v1.2.0"
}
include "root" {
  path = find_in_parent_folders()
}
locals {
  environment = "dev"
  region      = "us-east-1"
  name        = "demo-vpc"
}
inputs = {
  name = local.name
  cidr_block  = "10.0.0.0/16"
  azs  = ["us-east-1a", "us-east-1b"]
  tags = {
    Environment = local.environment
    ManagedBy = "terragrunt"
  }
}
dependency "network" {
  config_path = "../network"
}
remote_state {
  backend = "s3"
  config = {
    bucket  = "my-terraform-state-bucket"
    key     = "${path_relative_to_include()}/terraform.tfstate"
    region  = local.region
    encrypt = true
    dynamodb_table = "terraform-locks"
  }
}
```

- TERRAFORM MODULE SOURCE — Where the Terraform code lives.
- INCLUDE PARENT SETTINGS — Inherit common configuration.
- LOCALS — Reusable values.
- INPUTS — Variables passed to the module.
- DEPENDENCY — Read outputs from other stacks.
- REMOTE STATE — Store state securely in the backend.

### 6. WHY THIS MATTERS

- One place for common settings
- Easier environment and account management
- Consistent, reliable infrastructure deployments
- Scalable for teams and organizations

### 7. JUNIOR ENGINEER TIP

Terragrunt does not replace Terraform.

You still need to understand modules, providers, state, and cloud concepts.

Terragrunt helps you organize and scale them the right way.

VERIQTA

# BUILD YOUR FIRST TERRAGRUNT PROJECT

## FROM MODULE TO INFRASTRUCTURE IN MINUTES



### 1 CREATE A REUSABLE TERRAFORM MODULE

Build the infrastructure logic once so it can be reused across environments.

### 2 CREATE A TERRAGRUNT LIVE CONFIGURATION

Create a folder for your environment and add terragrunt.hcl.

### 3 POINT TERRAGRUNT TO THE MODULE

Use the terraform block and source to reference your module.

### 4 PASS INPUTS

Define inputs to customize the module for this environment.

### 5 INITIALIZE INFRASTRUCTURE

terragrunt init downloads modules and prepares the backend.

### 6 GENERATE A PLAN

terragrunt plan shows the changes that will be made.

### 7 APPLY INFRASTRUCTURE

terragrunt apply deploys the infrastructure.

### 8 INSPECT THE RESULTING RESOURCES

Verify resources in your cloud provider.

### 9 TERRAFORM CODE VS LIVE INFRASTRUCTURE CONFIG

Module = reusable code.
Terragrunt = environment configuration and execution.

## EXAMPLE: SIMPLE AWS S3 MODULE

### MODULE STRUCTURE

```
modules/
└── s3-bucket/
    ├── main.tf
    ├── variables.tf
    └── outputs.tf
```

modules/s3-bucket/main.tf

```
resource "aws_s3_bucket" "this" {
  bucket = var.bucket_name
  acl    = "private"
  tags = var.tags
}

resource "aws_s3_bucket_versioning"
 "this" {
  bucket = aws_s3_bucket.this.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

modules/s3-bucket/variables.tf

```
variable "bucket_name" {
  type        = string
  description = "Name of the S3 bucket"
}

variable "tags" {
  type    = map(string)
  default = {}
}
```

modules/s3-bucket/outputs.tf

```
output "bucket_id" {
  value  = aws_s3_bucket.this.id
}
```

## LIVE CONFIGURATION EXAMPLE

### LIVE FOLDER STRUCTURE

```
live/
└── dev/
    └── s3-bucket/
        └── terragrunt.hcl
```

live/dev/s3-bucket/terragrunt.hcl

```
terraform {
  source = "../../../../modules/s3-bucket"
}
inputs = {
  bucket_name     = "my-company-dev-bucket"
  tags            = {
    Environment       = "dev"
    Project           = "my-app"
    ManagedBy         = "terragrunt"
  }
}
```

### WHAT THIS DOES

1. Points to the reusable module
2. Passes environment-specific values
3. Terragrunt generates the Terraform execution context

ONE MODULE.
MANY ENVIRONMENTS.

## RUN TERRAGRUNT COMMANDS

| terragrunt init | terragrunt plan | terragrunt apply | terragrunt output | terragrunt destroy |
|---|---|---|---|---|
| Initialize the working directory and download modules. | Preview the changes that will be applied. | Apply the changes and create resources. | View outputs from your infrastructure. | Destroy resources when no longer needed. |

## TERRAFORM CODE vs LIVE INFRASTRUCTURE CONFIGURATION

### TERRAFORM MODULE (REUSABLE CODE)

- Defines WHAT to create
- Provider-agnostic logic
- Stored in modules/ or a shared repo
- Reused by many environments

WORK TOGETHER

MODULE + TERRAGRUNT = INFRASTRUCTURE

### TERRAGRUNT LIVE CONFIG (EXECUTION)

- Defines HOW and WHERE to deploy
- Environment, account, region, inputs
- Stores state and manages dependencies
- Executes Terraform safely at scale

### JUNIOR ENGINEER TIP

Start small. One module, one environment. Understand each step before scaling your infrastructure.
Terragrunt makes Terraform easier to manage, but your Terraform knowledge makes you effective.

# DRY INFRASTRUCTURE
## WITH INCLUDE AND PARENT CONFIGURATION

WRITE ONCE. USE EVERYWHERE.

### 1. WHAT DRY MEANS

**DRY = Don't Repeat Yourself**

DRY is a principle that reduces duplication by keeping common configuration in one place and reusing it everywhere.

DRY

- Less duplication
- Easier to maintain
- Fewer mistakes
- Consistent settings
- Safer changes

### 2. ROOT (PARENT) vs CHILD CONFIGURATION

**PARENT (ROOT)**

Contains shared settings used by all children.

- live/
  - terragrunt.hcl (root configuration)

**CHILD CONFIGURATIONS**

Each child inherits from the parent.

- dev/
  - app1/
    - terragrunt.hcl
- staging/
  - app1/
    - terragrunt.hcl
- prod/
  - app1/
    - terragrunt.hcl

Children focus only on what is different.

### 3. INCLUDE BLOCK

The include block tells Terragrunt where to find the parent configuration.

```
include "root" {
  path = find_in_parent_folders()
}
```

### 4. find_in_parent_folders()

This function automatically looks for the nearest parent terragrunt.hcl in the directory tree.

```
include "root" {
  path = find_in_parent_folders()
}
```

NO HARDCODED PATHS.
TERRAGRUNT FINDS THE PARENT FOR YOU.

### 5. EXAMPLE: PARENT AND CHILD CONFIGURATION

**PARENT: live/terragrunt.hcl**

```
terraform {
  source = "git::ssh://git@github.com/org/
  modules.git//vpc?ref=v1.0.0"
}
locals {
  region = "us-east-1"
  environment = "shared"
  tags = {
    ManagedBy   = "terragrunt"
    Owner       = "platform-team"
  }
}

inputs = {
  region = local.region
  default_tags = local.tags
}
```

SHARED SETTINGS LIVE HERE

**CHILD: live/dev/app1/terragrunt.hcl**

```
include "root" {
  path = find_in_parent_folders()
}

inputs = {
  environment = "dev"
  vpc_cidr    = "10.0.0.0/16"
  enable_nat  = true
}
```

ONLY DIFFERENCES HERE

### 6. WHAT GETS INHERITED?

- terraform block
- remote_state block
- locals
- inputs
- generate blocks
- providers (if generated)
- extra_arguments

All inherited settings are merged and passed to Terraform.

### 7. EXPOSING PARENT CONFIGURATION

You can expose values from the parent to children using:

```
include "root" {
  path   = find_in_parent_folders()
  expose = true
}
```

Then access in the child:

```
${include.root.locals.region}
```

### 8. AVOID COPY-AND-PASTE INFRASTRUCTURE

**BAD (DUPLICATED)**

Copying backend, providers, and inputs in every environment.

staging prod

**GOOD (DRY)**

Keep common settings in the parent and reuse everywhere.

PARENT

staging prod

### 9. COMMON INHERITANCE MISTAKES

**OVERRIDING UNINTENTIONALLY**
Redefining a block in the child replaces the parent completely.

**NOT UNDERSTANDING MERGE BEHAVIOR**
Maps merge, but lists and blocks may override.

**INCORRECT PATH USAGE**
Hardcoding paths breaks portability. Use find_in_parent_folders().

**TOO MUCH COMPLEXITY**
Deep inheritance chains can become hard to debug.

**NOT REVIEWING MERGED CONFIG**
Always run plan and check the final context Terragrunt generates.

### JUNIOR ENGINEER TIP

Terragrunt helps you stay DRY, consistent, and scalable. Understand inheritance, always inspect the final plan, and keep your parent configuration clean and minimal.

# LOCALS, INPUTS, AND ENVIRONMENT CONFIGURATION

## MAKE YOUR TERRAGRUNT CONFIGURATIONS DRY, FLEXIBLE, AND POWERFUL

### 1. TERRAGRUNT LOCALS

Locals let you define reusable values inside terragrunt.hcl.

```
locals {
  project     = "platform"
  environment = "dev"
  region      = "us-east-1"
  tags = {
    Project     = local.project
    Environment = local.environment
    ManagedBy   = "terragrunt"
  }
}
```

Use locals to build consistent values across your configuration.

### 2. INPUTS

Inputs pass values from Terragrunt to your Terraform modules.

```
inputs = {
  environment = local.environment
  region      = local.region
  vpc_cidr    = "10.0.0.0/16"
  tags        = local.tags
}
```

These values become Terraform variables inside your module.

### 3. TERRAFORM VARIABLES

Terraform variables receive the inputs from Terragrunt.

```
variable "environment" {
  type = string
}

variable "region" {
  type = string
}
```

The module stays reusable. Terragrunt provides the environment-specific values.

### 4. READING CONFIGURATION

Use read_terragrunt_config to read values from another Terragrunt configuration.

```
locals {
 root_cfg = read_terragrunt_config(
   find_in_parent_folders("root.hcl")
 )
 account_id = local.root_cfg.locals.account_id
}
```

This avoids duplication and keeps values in one place.

### 5. ENVIRONMENT-SPECIFIC VALUES

Define values for each environment.

| DEV | STAGING | PROD |
|---|---|---|
| environment = "dev" | environment = "staging" | environment = "prod" |
| instance_type = "t3.micro" | instance_type = "t3.small" | instance_type = "m6i.large" |
| min_size = 1 | min_size = 2 | min_size = 3 |
| max_size = 2 | max_size = 3 | max_size = 6 |

Each environment gets the right configuration without changing the module code.

### 6. ACCOUNT-SPECIFIC VALUES

Manage different cloud accounts.

**ACCOUNT A**

```
account_id = "111111111111"
role_arn   = "arn:aws:iam::
             111111111111:role/DeployRole"
```

**ACCOUNT B**

```
account_id = "222222222222"
role_arn   = "arn:aws:iam::
             222222222222:role/DeployRole"
```

**ACCOUNT C**

```
account_id = "333333333333"
role_arn   = "arn:aws:iam::
             333333333333:role/DeployRole"
```

Keeps credentials, roles, and limits isolated per account.

### 7. REGION-SPECIFIC VALUES

Deploy to multiple regions easily.

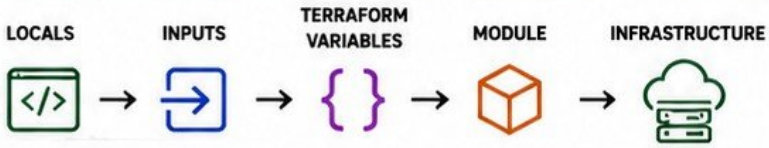
**us-east-1**
azs = ["us-east-1a", "us-east-1b", "us-east-1c"]

**us-west-2**
azs = ["us-west-2a", "us-west-2b", "us-west-2c"]

**eu-west-1**
azs = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]

Region values control availability zones, endpoints, and regional behavior.

### 8. VALUES FLOW DIAGRAM




| Define reusable values in terragrunt.hcl | Pass values to Terraform modules | Variables in modules receive the values | Module uses variables to create resources | Actual cloud infrastructure is created |
|---|---|---|---|---|

Terragrunt controls configuration and orchestration.
Terraform modules control infrastructure.

### 9. KEEPING CONFIGURATION UNDERSTANDABLE

- Centralize common values in root.hcl
- Use locals for computed or reusable values
- Keep environments, accounts, and regions separated
- Avoid long and complex inputs blocks
- Use clear naming and folder structure
- Prefer read_terragrunt_config over duplication
- Document important values and assumptions
- Review changes before applying
- Keep modules small and reusable

**SIMPLE, CONSISTENT, AND DRY = SAFE AT SCALE**

### 10. JUNIOR ENGINEER TIP

Terragrunt does not replace Terraform. It helps you organize, reuse, and manage Terraform configurations across environments, accounts, and regions without duplication and confusion. You still need to understand Terraform modules, variables, providers, and state.

# REMOTE STATE AND BACKEND MANAGEMENT

## KEEP YOUR INFRASTRUCTURE STATE SAFE, SHARED, AND RELIABLE

### 1. WHY TERRAFORM STATE MATTERS


Terraform state is the source of truth for your infrastructure. It maps real-world resources to your configuration. Without it, Terraform cannot track, update, or destroy resources safely.

### 2. LOCAL vs REMOTE STATE

| LOCAL STATE | REMOTE STATE |
|---|---|
| Stored on your machine | Stored in cloud storage |
| Not shared with team | Shared with team safely |
| Risk of state loss | State locking enabled |
| No locking | Versioned & backed up |
| Not for production | Recommended for production |

### 3. TERRAGRUNT REMOTE_STATE BLOCK

```
remote_state {
  backend = "s3"

  config = {
    bucket         = "my-terraform-state"
    key            = "env:/path/to/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "my-terraform-locks"
    encrypt        = true
  }
generate = {
    path      = "backend.tf"
    if_exists = "overwrite_terragrunt"
  }
}
```

### 4. GENERATING BACKEND CONFIGURATION

Terragrunt can automatically generate the backend.tf file that Terraform needs.

This keeps backend configuration DRY and consistent across all environments.

Generated backend.tf

```
terraform {
  backend "s3" {
    bucket         = "my-terraform-state"
    key            = "env:/path/to/terraform.tfstate"
    region         = "us-east-1"
    dynamodb_table = "my-terraform-locks"
    encrypt        = true
  }
}
```

### 5. ENVIRONMENT ISOLATION

| DEV | STAGING | PROD |
|---|---|---|
| Bucket: my-terraform-state<br>Key: dev/network/terraform.tfstate | Bucket: my-terraform-state<br>Key: staging/network/terraform.tfstate | Bucket: my-terraform-state<br>Key: prod/network/terraform.tfstate |

Each environment has its own state file (key) and isolation.
No one environment can overwrite another.

### 6. STATE LOCKING

DynamoDB (Table)

State locking prevents multiple users or pipelines from running Terraform operations at the same time, avoiding corruption.

Terragrunt uses DynamoDB (or your chosen backend lock mechanism) to lock the state.

Only one operation at a time!

### 7. STATE NAMING BEST PRACTICES

- Use clear, consistent naming
- Include environment, component, and region
- Avoid spaces and special characters
- Example Pattern:

<env>/<account>/<region>/<component>/terraform.tfstate

Example:

prod/11111111111/us-east-1/network/terraform.tfstate

### 8. AWS S3 BACKEND EXAMPLE


S3 Bucket (State Storage) | DynamoDB (State Locking)

- S3 stores the state file
- DynamoDB locks the state
- Enable versioning on S3
- Enable server-side encryption

### 9. PROTECTING STATE FILES

- Store state in private buckets
- Enable S3 versioning
- Enable server-side encryption (SSE)
- Restrict IAM access (least privilege)
- Enable bucket logging and monitoring
- Regular backups and lifecycle policies

### 10. PRODUCTION RISK: STATE CONTAINS SENSITIVE INFORMATION

State files can contain sensitive data such as:

- Resource IDs and ARNs
- IP addresses and endpoints
- Secrets (db passwords, tokens, keys) if outputs are not masked
- Infrastructure topology

**TREAT STATE FILES LIKE SECRETS. PROTECT THEM!**

# GENERATE BLOCKS AND PROVIDER CONFIGURATION

## AUTOMATE COMMON FILES, STAY DRY, AND KEEP CONFIGURATION CONSISTENT

### 1. WHY TEAMS GENERATE COMMON FILES

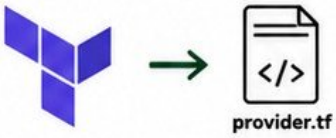
- Avoid repeating the same backend and provider configuration in every folder.
- Keep configurations DRY (Don't Repeat Yourself).
- Ensure consistency across all environments and accounts.
- Reduce human error and maintenance overhead.
- Focus on infrastructure logic, not boilerplate.

### 2. THE GENERATE BLOCK

```
generate "provider" {
  path      = "provider.tf"
  if_exists = "overwrite_terragrunt"
  contents  = <<EOF
    provider "aws" {
      region = var.aws_region
      default_tags
          tags = {
            ManagedBy = "terragrunt"
        }
    }
  EOF
```

Terragrunt writes this content to provider.tf before Terraform runs.

### 3. PROVIDER CONFIGURATION




- Define providers once in a central place.
- Generate provider.tf in each unit.
- All environments use the same provider logic.
- Change once, update everywhere.

### 4. BACKEND GENERATION

```
generate "backend" {
  path      = "backend.tf"
  if_exists = "overwrite_terragrunt"
  contents  = <<EOF
  terraform {
   backend "s3" {
     bucket         = "my-terraform-state"
     key            = "env/${local.env}/
     ${path_relative_from_include()}
                      /terraform.tfstate"
     region         = "us-east-1"
     dynamodb_table = "my-terraform-locks"
     encrypt        = true
   }
  EOF
```

Backend configuration is generated automatically for every environment.

### 5. SHARED TERRAFORM FILES

You can generate any Terraform file, not just provider.tf or backend.tf.

- Versions file
- Required providers
- Provider aliases
- Common locals
- Terraform settings

provider.tf backend.tf versions.tf locals.tf ...

All generated at runtime by Terragrunt.

### 6. IF_EXISTS OPTIONS

| OPTION | BEHAVIOR |
|---|---|
| overwrite | Always overwrite the file if it exists. |
| overwrite_terragrunt | Overwrite only if the file was previously generated by Terragrunt. |
| keep | Keep the existing file. Do not overwrite. |
| error | Throw an error if the file already exists. |

Recommended: overwrite_terragrunt
Safe and predictable for most teams.

### 7. AVOIDING DUPLICATED PROVIDER BLOCKS

| WITHOUT GENERATE | WITH GENERATE |
|---|---|
| provider.tf ✗ | provider.tf ✓ |
| provider.tf ✗ | provider.tf ✓ |
| provider.tf ✗ | provider.tf ✓ |
| Duplicate, inconsistent, hard to maintain. | Consistent, DRY, easy to manage. |

### 8. GENERATED vs SOURCE-CONTROLLED FILES

**GENERATED FILES (Not committed)**

- Created by Terragrunt at runtime
- Should NOT be edited manually
- Ignored by .gitignore
- Regenerated on every run

**SOURCE-CONTROLLED FILES (Committed to Git)**

- terragrunt.hcl files
- Modules and code
- Documentation
- Shared configurations

Rule: Never commit generated files.
Commit only Terragrunt configs and modules.

### 9. WHEN GENERATION BECOMES TOO COMPLICATED

- Too many generated files
- Complex logic in generate blocks
- Hard to debug and understand
- Hidden behavior across folders
- Hard for new engineers to follow

Keep it simple.
Generate only what truly needs to be standardized.

### 10. JUNIOR ENGINEER TIP

**TERRAGRUNT GENERATE BLOCKS SAVE TIME AND PREVENT DUPLICATION, BUT OVERUSE CAN CREATE MAGIC. KEEP IT SIMPLE, CLEAR, AND DOCUMENTED.**

# ORGANIZING DEV, STAGING, AND PRODUCTION

## STRUCTURE YOUR TERRAGRUNT REPOSITORIES FOR SCALE, CLARITY, AND SAFE OPERATIONS

### 1. INFRASTRUCTURE REPOSITORY STRUCTURES

**MODULES REPOSITORY**

- Reusable Terraform modules
- Versioned and published
- Shared across environments and accounts

Example: github.com/org/terraform-modules

**LIVE INFRASTRUCTURE REPOSITORY**

- Terragrunt configurations
- Environment, account and region specific
- Contains no reusable module code

Example: github.com/org/live-infra

### 2. STRUCTURE DIMENSIONS

**ACCOUNT STRUCTURE**
Organize by AWS/GCP accounts or tenants.

**REGION STRUCTURE**
Group resources by the region they are deployed in.

**ENVIRONMENT STRUCTURE**
Separate environments: Dev, Staging, Production.

**COMBINATION**
Most teams use all three dimensions together.

### 3. ENVIRONMENT STRUCTURE

**DEV**

- For development and testing
- Lower cost, smaller scale
- Frequent changes allowed

**STAGING**

- Pre-production testing
- Mirrors production closely
- Validates changes before prod

**PRODUCTION**

- Live workloads and real users
- High availability and reliability
- Change controlled and monitored

### 4. KEEPING ENVIRONMENTS ISOLATED

**SEPARATE STATE**
Each environment must have its own remote state.

**SEPARATE ACCOUNTS**
Use different accounts for strong isolation.

**SEPARATE NETWORKS**
VPCs, subnets, and resources should not overlap.

**SEPARATE CREDENTIALS**
Use environment-specific IAM roles and credentials.

**SEPARATE PIPELINES**
Different pipelines, approvals, and deployment rules.

**BENEFITS**

- Prevents accidental cross-environment changes
- Improves security and compliance
- Safer deployments and testing
- Clear ownership and boundaries
- Easier troubleshooting and management

**WARNING**
Sharing state, accounts, or networks between environments is one of the biggest risks infrastructure teams can take.

### 5. EXAMPLE PRODUCTION DIRECTORY TREE

```
live-infra/
├── terragrunt.hcl          # Root configuration
├── _common/                # Shared configurations
│   ├── terragrunt.hcl
│   └── locals.hcl
├── accounts/
│   ├── prod/
│   │   └── us-east-1/
│   │       ├── terragrunt.hcl
│   │       ├── vpc/
│   │       ├── networking/
│   │       ├── database/
│   │       ├── eks/
│   │       ├── applications/
│   │       └── monitoring/
│   │   └── us-us-west-2/
│   │       ├── terragrunt.hcl
│   │       └ ...
├── staging/
│   └── us-east-1/   ...
└── dev/
    └── us-east-1/   ...
```

Root and common settings live here

Production account resources organized by region

Each folder is a Terragrunt unit (manages a module)

Folder · File · Reference · ... More items

### 6. CHOOSING A STRUCTURE TEAMS CAN NAVIGATE SAFELY

- Keep it SIMPLE and consistent
- Use a structure your team understands
- Scale the structure as the organization grows
- Document the structure and decisions
- Prioritize readability over clever structure
- Review and refactor as needed
- Design for engineers, not just for tools

### 7. RECOMMENDED HIGH-LEVEL STRUCTURE

**MODULES REPOSITORY**
Reusable Terraform modules (versioned)

→

**LIVE INFRASTRUCTURE REPOSITORY**
Terragrunt configurations by account, region, and environment

→

**ENVIRONMENTS**
Dev, Staging, Production (fully isolated)

→

**INFRASTRUCTURE**
Secure, reliable, scalable resources in the cloud

### 8. JUNIOR ENGINEER TIP

**THE BEST TERRAGRUNT STRUCTURE IS NOT THE MOST COMPLEX.
IT IS THE ONE YOUR TEAM CAN UNDERSTAND, SCALE, AND TRUST.**

# TERRAGRUNT DEPENDENCIES AND INFRASTRUCTURE RELATIONSHIPS

## 1 WHY COMPONENTS DEPEND ON EACH OTHER

**Infrastructure is not isolated. Resources rely on outputs from other resources.**

**Examples:**

- EKS needs VPC, subnets, and security groups
- RDS needs VPC and security groups
- Applications need security groups and DNS
- A change in one layer can impact others

## 2 HOW TERRAGRUNT HANDLES DEPENDENCIES

- Uses dependency blocks
- Reads Terraform outputs
- Passes outputs as inputs to other modules

**Example dependency block**

```
dependency "vpc" {
  config_path  = "../vpc"
}

inputs = {
  vpc_id       = dependency.vpc.outputs.vpc_id
  private_subnets  =
      = dependency.vpc.outputs.private_subnets
}
```

## 3 TERRAFORM OUTPUTS AND PASSING THEM BETWEEN MODULES

**In the dependency module (producer)**

```
# outputs.tf
output "vpc_id" {
  value = aws_vpc.main.id
}

output "private_subnets" {
  value = aws_subnet.private[*].id
}
```

OUTPUTS → CONSUMED BY OTHER MODULES

**In the consuming module**

```
inputs = {
  vpc_id  = dependency.vpc.outputs.vpc_id
  private_subnets  = dependency.vpc.outputs.private_subnets
}
```

## 4 COMMON INFRASTRUCTURE RELATIONSHIPS

**VPC** — Provides network foundation

→ **EKS** — Needs VPC, subnets, security groups

→ **RDS** — Needs VPC and security groups

→ **SECURITY GROUPS** — Control traffic for apps, databases, etc.

→ **APPLICATIONS** — Need network, security groups, and services

## 5 DEPENDENCY OUTPUTS

OUT

Outputs are the contract between modules.

Keep outputs:

- Small and focused
- Stable and versioned
- Not containing sensitive data

## 6 MOCK OUTPUTS

MOCK

Used when the dependency has not been applied yet. Allows you to run plan without real values.

```
dependency "vpc" {
  config_path  = "../vpc"

  mock_outputs = {
    vpc_id  = "vpc-123456"
    private_subnets  = ["subnet-aaaa","subnet-bbb"]
  }
  mock_outputs_allowed_terrafoom_commands = ["init","validate","plan"]
}
```

## 7 COMMON DEPENDENCY FAILURES

**WRONG CONFIG PATH** — Terragrunt cannot find the dependency configuration.

**MISSING OUTPUT** — The expected output does not exist in the dependency module.

**STATE NOT FOUND** — Remote state not initialized or backend misconfigured.

**APPLY ORDER ISSUES** — Running a module before its dependency exists.

**PERMISSION ERRORS** — Insufficient access to read state or outputs from dependency.

## JUNIOR ENGINEER TIP

Understand your dependencies before you run apply.
Always plan from the top of the dependency chain.
Make small, verified changes and let Terragrunt manage the order.

**GOLDEN RULE**

Good dependencies make infrastructure reliable.
Bad dependencies make production fragile.

VERIQTA



# BUILDING AN INFRASTRUCTURE DEPENDENCY GRAPH

## VISUALIZE. ORDER. DEPLOY. SAFELY.

### 1 INFRASTRUCTURE AS A GRAPH

Infrastructure components depend on each other.
A dependency graph helps you understand relationships, determine order, and prevent failures.

### 2 CORE COMPONENTS (EXAMPLE STACK)

**VPC**
Network foundation

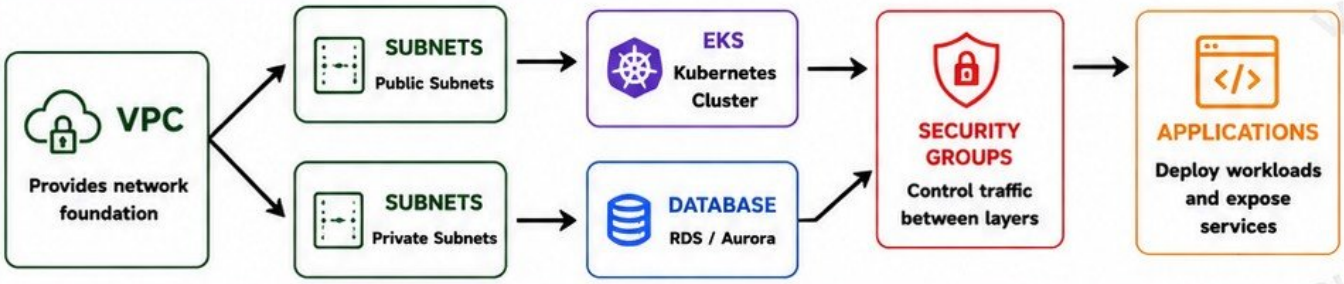
**SUBNETS**
Segments within the VPC

**DATABASE**
Stateful data layer

**EKS**
Kubernetes cluster

**APPLICATIONS**
Workloads and services

### 3 INFRASTRUCTURE DEPENDENCY GRAPH (EXAMPLE)




### 4 DEPENDENCY ORDERING MATTERS

1. VPC
2. SUBNETS
3. DATABASE
4. EKS
5. SECURITY GROUPS
6. APPLICATIONS

Deploy in the correct order to avoid failures and downtime.
Lower layers must exist before higher layers can be created.

### 5 RUNNING DEPENDENT INFRASTRUCTURE SAFELY

- Understand the dependency graph
- Plan changes from bottom to top
- Validate outputs before consuming
- Use Terragrunt dependencies
- Run targeted plans
- Review impacts before apply
- Automate with CI/CD guardrails

Small changes in the foundation can affect multiple dependent resources.

**CHECK. PLAN. VALIDATE. APPLY.**

### 6 WHY DEPLOYMENT ORDER MATTERS

- Resources fail if dependencies are missing
- Partial infrastructure causes broken states
- Rework is time-consuming and costly
- Downtime impacts production users
- Harder to troubleshoot and recover

### 7 PRODUCTION CONNECTION

**Changing foundational infrastructure can affect everything above it.**

**EXAMPLES:**

- Changing VPC CIDR can break EKS, RDS, and apps
- Deleting subnets can disrupt running workloads
- Modifying security groups can cut off traffic
- Altering databases can impact applications

### 8 JUNIOR ENGINEER TIP

Always map dependencies before you deploy.
Understand the full impact of a change.
Infrastructure is connected—treat it that way.

**Good engineers build. Great engineers plan and protect.**

VERIQTA



# RUNNING MULTIPLE TERRAGRUNT UNITS SAFELY

## OPERATE AT SCALE. PLAN CAREFULLY. APPLY SAFELY.

### 1 OPERATING ACROSS MULTIPLE CONFIGURATIONS

Terragrunt lets you manage many units (configurations) in one repository.

Each unit represents a piece of infrastructure.

**EXAMPLES OF UNITS**

- Network (VPC, Subnets)
- Database (RDS, Aurora)
- Compute (EKS, ASG)
- Security (IAM, SG)
- Applications
- Observability

### 2 STACK-AWARE EXECUTION CONCEPTS

Terragrunt understands dependencies between units using dependency blocks.

It builds a dependency graph and runs units in the correct order.

**KEY IDEAS**

- Know what depends on what
- Run in dependency order
- Only run what is needed
- Avoid breaking changes to core layers

### 3 PLANNING MULTIPLE INFRASTRUCTURE UNITS

Generate plans for many units before making any changes.

Review all changes in one place.

**COMMON COMMANDS**

```
terragrunt run-all plan

terragrunt run-all plan \
  --terragrunt-include-dir "network/**"

terragrunt run-all plan \
  --terragrunt-exclude-dir "**/prod/**"

terragrunt run-all plan \
  --graph-dependencies
```

### 4 APPLYING COORDINATED CHANGES

Apply changes across multiple units in the correct order to maintain stability.

**COMMON COMMANDS**

```
terragrunt run-all apply

terragrunt run-all apply \
  --terragrunt-include-dir "envs/dev/**"

terragrunt run-all apply \
  --queue-include-external
```

### 5 EXECUTION ORDERING

Terragrunt determines execution order based on dependencies.

TYPICAL ORDER → NETWORK (VPC, SUBNETS) → DATABASE (RDS, AURORA) → EKS / COMPUTE → SECURITY (SG, IAM) → APPLICATIONS & SERVICES

### 6 DEPENDENCY AWARENESS

Terragrunt uses dependency blocks to understand relationships.

**BENEFITS**

- Prevents broken states
- Ensures outputs are available
- Avoids applying in wrong order
- Reduces manual coordination

### 7 PARALLEL EXECUTION CONSIDERATIONS

Terragrunt can run independent units in parallel, which speeds up execution.

But not everything should run in parallel.

**CONSIDERATIONS**

- Respect dependencies
- Limit provider API throttling
- Avoid overwhelming state backends
- Test at small scale first

### 8 LIMITING BLAST RADIUS

Change one layer at a time.

Limit the impact of a mistake.

**BEST PRACTICES**

- Apply in environments (dev → staging → prod)
- Use `--terragrunt-include-dir`
- Avoid run-all apply on entire repo
- Review plans carefully

### 9 WHY INSPECT PLANS BEFORE LARGE-SCALE EXECUTION

Plans show what will change.
They reveal:

- Resources to be created, updated, or destroyed
- Unexpected replacements
- Risky changes to foundational resources
- Ripple effects across dependent systems

**NEVER APPLY BLINDLY.**

A small mistake can have a huge impact.

### 10 DANGEROUS BULK OPERATIONS

**HIGH-RISK COMMANDS**

```
terragrunt run-all apply
(on the entire repository)
------------------------------
terragrunt run-all destroy
------------------------------
terragrunt run-all apply --terragrunt-non-interactive
```

### JUNIOR ENGINEER TIP

Terragrunt does not prevent bad decisions.
It only helps you run many units in the right order.
Understand your architecture. Plan carefully. Make safe changes.

**GOOD ENGINEERS AUTOMATE. GREAT ENGINEERS OPERATE WITH AWARENESS.**

# HOOKS AND EXTRA TERRAFORM ARGUMENTS

## EXTEND, VALIDATE, AND CONTROL YOUR INFRA WORKFLOWS

### 1 WHY THIS MATTERS

Terragrunt lets you run commands before, after, or on error.
You can also pass common Terraform arguments consistently.

- Automate checks
- Enforce standards
- Reduce human mistakes
- Keep workflows consistent

### 2 TERRAGRUNT HOOKS OVERVIEW

**BEFORE HOOKS**

- Run before a command (executes before init, plan, apply, etc.)
- Use for validation, checks, formatting
- Can block execution

**AFTER HOOKS**

- Run after a command (executes after success)
- Use for notifications, logging, summaries
- Does not affect the result

**ERROR HOOKS**

- Run only when a command fails
- Use for alerts, cleanup, debugging
- Helps capture failures

### 3 HOOKS EXAMPLE

**EXAMPLE: RUN CHECKS BEFORE PLAN**

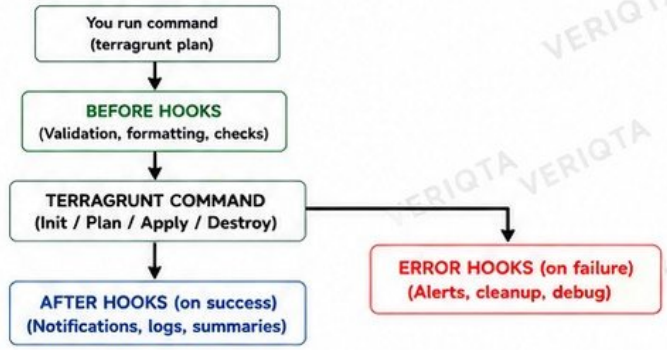
```
terraform {
  source = "git::ssh://github.com/org/module.git"
}

hooks {
  before_hook "validate" {
    commands = ["plan", "apply", "destroy"]
    execute  = ["bash", "-c", "terraform fmt -check -diff && terraform validate"]
  }

  after_hook "notify" {
    commands = ["apply"]
    execute  = ["bash", "-c", "echo '✅ Apply completed for ${PWD}'"]
  }

  error_hook "on_error" {
    commands = ["plan", "apply", "destroy", "init"]
    execute  = ["bash", "-c", "echo '❌ Terragrunt command failed in ${PWD}'"]
  }
}
```

**HOOK EXECUTION FLOW**




### 4 EXTRA_ARGUMENTS: PASS COMMON TERRAFORM ARGUMENTS

Use extra_arguments to pass flags to Terraform automatically.

**EXAMPLE**

```
terraform {
  extra_arguments "common_args" {
    commands  = get_terraform_commands_that_need_vars()
    arguments = [
        "-lock-timeout=10m",
        "-parallelism=10",
        "-input-false",
        "-compact-warnings"
    ]
  }
}
```

### 5 COMMON USE CASES

- **VALIDATION WORKFLOWS** — Run fmt, validate, tflint, terraform-docs before plan.
- **NOTIFICATIONS** — Send Slack/Email on success or failure.
- **AUDIT & LOGGING** — Capture who ran what, where, and when.
- **POLICY ENFORCEMENT** — Stop execution if security or compliance checks fail.
- **CLEANUP ON FAILURE** — Clean temporary files or release locks.
- **VISIBILITY** — Print summaries, outputs, or links after apply.

### 6 WHEN HOOKS ARE USEFUL

- Enforcing standards automatically
- Preventing bad changes early
- Improving team consistency
- Adding observability and notifications
- Reducing repetitive manual steps

### 7 WHY HOOKS SHOULD NOT BECOME HIDDEN AUTOMATION

- Harder to troubleshoot
- Logic becomes invisible to the team
- Makes workflows unpredictable
- Can hide failures or mask problems
- Keep hooks simple, visible, and documented

### 8 BEST PRACTICES

- Keep hooks small and focused
- Document every hook and why it exists
- Avoid complex scripts in hooks
- Use hooks for guardrails, not business logic
- Review hooks like you review code

### JUNIOR ENGINEER TIP

Hooks and extra_arguments help you enforce good habits and save time.
But never rely on them blindly—always understand what runs and why.

**AUTOMATE GUARDRAILS. NOT BUSINESS LOGIC.**

# SECRETS, IAM, AND PRODUCTION SECURITY

## PROTECT YOUR INFRASTRUCTURE. PROTECT YOUR BUSINESS.

### 1 NEVER HARDCODE CREDENTIALS

Never put credentials, API keys, tokens, or secrets directly in your Terragrunt or Terraform configuration.

**BAD EXAMPLE (DO NOT DO THIS)**

```
inputs = {
  db_password = "MySecret123"
  aws_access_key = "AKIA..."
}
```

### 2 ENVIRONMENT VARIABLES

Use environment variables to pass sensitive values at runtime.

**EXAMPLE**

```
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
export TF_VAR_db_password=...
terragrunt plan
```

### 3 CLOUD AUTHENTICATION

Use your cloud provider's official authentication methods (AWS CLI, Azure CLI, gcloud, OIDC).

**EXAMPLE (AWS CLI)**

```
aws configure
aws sts get-caller-identity
```

### 4 IAM ROLES

Use IAM roles instead of long-lived keys. Let your environment assume the right role.

**EXAMPLE (AWS)**

```
aws sts assume-role \
 --role-arn arn:aws:iam::123456789012:role/
  TerragruntRole
 --role-session-name terragrunt
```

### 5 SHORT-LIVED CREDENTIALS

Use short-lived credentials to reduce the risk of key leakage and unauthorized access.

**BENEFITS**

- Automatically expire
- Lower security risk
- Ideal for CI/CD and teams

### 6 LEAST PRIVILEGE

Give only the minimum permissions required. Avoid using admin or full-access policies.

**PRINCIPLES**

- Minimum required permissions
- Resource-level access
- Separate roles per environment
- Regularly review permissions

### 7 PROTECTING STATE

Terraform state can contain sensitive information. Store it securely and restrict access.

**BEST PRACTICES**

- Use remote state (e.g. S3 + lock)
- Encrypt state at rest
- Restrict access with IAM
- Enable versioning
- Monitor access

### 8 SENSITIVE TERRAFORM OUTPUTS

Mark sensitive outputs to prevent them from being shown in the CLI, logs, or CI pipelines.

**EXAMPLE**

```
output "db_password" {
  value     = aws_db_instance.db.password
  sensitive = true
}
```

### 9 SECRET MANAGERS

Use a secret manager to store and manage secrets securely.

**COMMON OPTIONS**

- AWS Secrets Manager
- Azure Key Vault
- Google Secret Manager
- HashiCorp Vault

### 10 CI/CD IDENTITIES

Use secure identities in your CI/CD pipeline (e.g. OIDC) instead of static credentials.

**COMMON METHODS**

- GitHub Actions OIDC
- GitLab CI OIDC
- AWS IAM Roles for Service Accounts
- Workload identity federation

### 11 CROSS-ACCOUNT ACCESS

Use role assumption for cross-account access. Avoid sharing long-lived credentials between accounts.

**EXAMPLE**

```
provider "aws" {
  assume_role {
    role_arn = "arn:aws:iam::123456789012:role/
    CrossAccountRole"
  }
}
```

### 12 SECURITY REMINDER

**Terragrunt configuration belongs in Git, credentials do not.**

**KEEP YOUR INFRASTRUCTURE SECURE.**
**KEEP SECRETS OUT OF YOUR REPOSITORY.**

**JUNIOR ENGINEER TIP**

Security is part of your job. Always think about where secrets are stored, who can access them, and what happens if they are exposed.

**SECURE TODAY. STABLE TOMORROW.**

# TERRAGRUNT IN CI/CD

## AUTOMATE INFRASTRUCTURE DELIVERY THE RIGHT WAY

### 1 PULL REQUEST WORKFLOW

All changes start with a Pull Request (PR). It triggers automated checks before anything is applied.

### 2 CI RUNNERS

CI runners execute Terragrunt commands in isolated, repeatable environments.

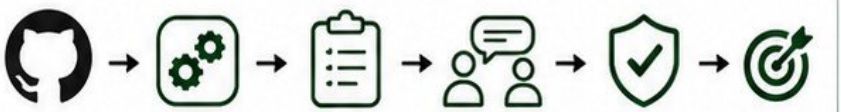
### 3 CLOUD AUTHENTICATION

Use secure, short-lived credentials. Authenticate with IAM Roles, OIDC, or Workload Identity.

### 4 CI PIPELINE STAGES

1. **FORMATTING**
   Run terragrunt hclfmt and terraform fmt to keep code style consistent.
2. **VALIDATION**
   Run terragrunt validate and tflint to catch errors early.
3. **TERRAGRUNT PLAN**
   Run terragrunt run-all plan to generate execution plans.
4. **PLAN REVIEW**
   Plans are posted on the PR for easy review and discussion.
5. **APPROVAL**
   Human approval is required before any apply step can run.
6. **TERRAGRUNT APPLY**
   On approval, run terragrunt run-all apply to provision or update infrastructure.
7. **VERIFY**
   Post-apply checks and notifications confirm successful deployment.

### 5 CI/CD PIPELINE DIAGRAM



| GIT | CI | PLAN | REVIEW | APPLY | VERIFY |
|---|---|---|---|---|---|
| Code pushed to repository | Pipeline triggered | Terragrunt run-all plan | Plan reviewed in PR | Terragrunt run-all apply | Verify resources and state |

**PIPELINE FLOW**

Git → CI → Plan → Review → Apply → Verify

### 6 ENVIRONMENT PROMOTION

**DEV**
Develop and test changes

↓

**STAGING**
Validate in staging environment

↓

**PRODUCTION**
Promote to production with control

### 7 PRODUCTION APPROVAL GATES

- Require explicit approvals
- Restrict who can approve
- Environment-specific protections
- Audit every change
- Break-glass only for emergencies

**NO APPROVAL. NO PRODUCTION DEPLOYMENT.**

### 8 BEST PRACTICES

- Always run format and validate
- Never apply from feature branches
- Review every plan carefully
- Use environment promotion, not direct changes
- Use least-privilege CI identities
- Keep state secure and separate per environment
- Pin Terragrunt and Terraform versions
- Use targeted applies only when necessary
- Document pipeline steps clearly
- Monitor pipeline runs and notifications
- Automate but never remove human oversight

**JUNIOR ENGINEER TIP**

Automation is powerful, but visibility and approvals make infrastructure safe.

**SECURITY REMINDER**

Terragrunt configuration belongs in Git. Credentials and secrets do not.

**AUTOMATE WITH CONFIDENCE. DEPLOY WITH CONTROL.**

# SAFE PRODUCTION CHANGES
## AND INFRASTRUCTURE GOVERNANCE

Small mistakes in infrastructure can cause big outages and high cost.
Governance, review, and discipline keep production safe.

### 1 PLAN BEFORE APPLY

PLAN → REVIEW → APPROVE → APPLY

- Always run terragrunt plan
- Inspect the full plan output
- Understand every change
- Never skip plan in production

**NO PLAN. NO APPLY.**

### 2 REVIEWING DESTRUCTIVE CHANGES

Look for actions that can **delete or replace resources.**

- Destroy
- Replace
- Create Before Destroy
- Update in-place (risk of downtime)
- Downgrade or drift
- State changes

If you don't understand the change, don't approve it.

### 3 RESOURCE REPLACEMENT

Many changes require resource replacement instead of in-place updates.

**COMMON EXAMPLES**

- Changing instance type
- Changing name of some resources
- Immutable resource properties
- Changing storage type
- Altering subnet CIDR
- Changing security group rules that force replacement

Replacement can cause downtime or data loss.

### 4 BLAST RADIUS

Every change has a blast radius.

Ask before you apply:

- What resources are affected?
- What systems depend on this?
- What is the worst case impact?
- Can this be rolled back?

**THE GOAL: SMALL, SAFE, AND ISOLATED CHANGES.**

### 5 ENVIRONMENT BOUNDARIES

DEV
STAGING
PRODUCTION

- Strictly separate environments
- No direct changes in production
- Promote through proper flow
- Different state, accounts, and permissions

**BOUNDARIES PREVENT ACCIDENTS.**

### 6 CHANGE APPROVAL

- Code review for all infrastructuce changes
- Peer review the plan
- Use approval gates in CI/CD
- Audit every change
- Document the reason for change

**NO REVIEW, NO DEPLOY.**

### 7 VERSION PINNING

- Pin Terraform and Terragrunt versions
- Avoid automatic upgrades
- Ensure the team uses the same versions
- Test before upgrading

**UNPINNED VERSIONS = UNPREDICTABLE BEHAVIOR.**

### 8 TERRAFORM/TERRAGRUNT COMPATIBILITY

- Check compatibility before upgrading
- Read release notes
- Test in lower environments
- Avoid major version jumps in prod
- Maintain a standard version matrix

**COMPATIBILITY ISSUES CAUSE FAILURES.**

### 9 MODULE VERSIONING

- Use versioned module sources
- Avoid pointing to "main" or "latest"
- Use semantic versions
- Update intentionally
- Track module changes

**VERSION EVERYTHING.**

### 10 PROTECTING PRODUCTION

- Least privilege access
- State stored securely with locking
- Protect production state bucket
- Use MFA and roles
- Monitor and audit access
- Limit who can apply changes

**SECURITY + GOVERNANCE = PRODUCTION SAFETY.**

### 11 ROLLBACK LIMITATIONS IN INFRASTRUCTURE

- Terraform can destroy, but cannot undo real-world impact
- Deleted data is not restored
- Recreated resources get new IDs
- Rollback may require manual steps
- State rollback ≠ Infrastructure rollback
- Some changes are irreversible

**ROLLBACK IS NOT A TIME MACHINE.**
**PREVENT > DETECT > RECOVER**

### 12 WHY INFRASTRUCTURE ROLLBACK DIFFERS FROM APPLICATION ROLLBACK

**APPLICATION ROLLBACK**

- Quick and simple
- Previous version deployed
- Minimal external dependency

**INFRASTRUCTURE ROLLBACK**

- May require destroy + recreate
- Data may be lost
- Dependencies must be handled
- Longer recovery time

**INFRASTRUCTURE NEEDS CARE, NOT JUST SPEED.**

### JUNIOR ENGINEER TIP

Governance is not about slowing you down.
It is about protecting the system, the data, and your team.
A good engineer plans, reviews, communicates, and understands the impact.

### REMEMBER

- Plan carefully
- Review everything
- Apply safely
- Learn continuously

# TROUBLESHOOTING TERRAGRUNT FAILURES

FIND THE ROOT CAUSE, FIX IT FAST, AND KEEP PRODUCTION SAFE.

## 1 READ THE ACTUAL ERROR FIRST

- Errors tell you what went wrong.
- Read the full message.
- Don't guess. Don't skip.
- The first error is usually the root cause.

**ALWAYS READ BEFORE YOU CHANGE ANYTHING.**

## 2 TERRAGRUNT ERROR vs TERRAFORM ERROR

### TERRAGRUNT ERRORS

- HCL syntax issues
- Invalid includes or paths
- Dependency block issues
- Remote state config problems
- Terragrunt command problems

### TERRAFORM ERRORS

- Provider / plugin errors
- Resource configuration issues
- Cloud API / permission errors
- State / lock errors
- Module / variable issues

## 3 COMMON FAILURE AREAS

**MODULE SOURCE FAILURES**
Wrong path, repo, ref, or auth

**MISSING VARIABLES**
Required inputs not provided

**DEPENDENCY FAILURES**
Dependency output not found or failed to apply

**REMOTE-STATE PROBLEMS**
State not found, access denied, or backend misconfigured

**AUTHENTICATION FAILURES**
Expired credentials, wrong role, or missing permissions

**PROVIDER ERRORS**
Provider bugs, wrong region, or unsupported features

**STATE LOCKING**
Another process is holding the lock

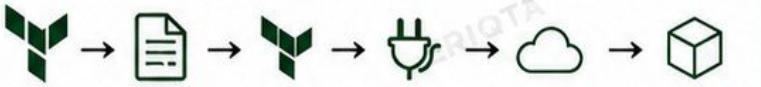
**INCORRECT PATHS**
Wrong working directory, include path, or file reference

## 4 DEBUGGING YOUR CONFIGURATION

- `Use terragrunt hclfmt --terragrunt-check`
- `Use terragrunt render to see generated files`
- `Use terragrunt validate-inputs`
- `Use terragrunt plan --terragrunt-log-level debug`
- `Check includes and find_in_parent_folders() paths`
- `Print localsand inputs to verify values`

## 5 TROUBLESHOOTING FLOW

Follow the path from high-level to low-level.



| TERRAGRUNT (Command & Orchestration) | CONFIGURATION (terragrunt.hcl, inputs, includes, dependencies) | TERRAFORM (Plan, Graph, State, Modules) | PROVIDER (AWS, Azure, GCP, etc.) | CLOUD API (Permissions, Limits, Quotas) | RESOURCE (Actual Infra Created/Updated) |
|---|---|---|---|---|---|

**TIP: FIX THE FIRST BROKEN LINK IN THE CHAIN. DON'T JUMP TO THE BOTTOM.**

## QUICK DEBUG COMMANDS

| | |
|---|---|
| `terragrunt plan` | See the full plan |
| `terragrunt plan -detailed-exitcode` | Use in CI/CD |
| `terragrunt apply` | Apply infrastructure |
| `terragrunt validate-inputs` | Validate inputs |
| `terragrunt render` | See generated Terraform |
| `terragrunt graph` | View dependency graph |
| `terragrunt hclfmt` | Format HCL files |
| `terragrunt --terragrunt-log-level debug` | Debug Terragrunt |
| `terraform providers` | Check providers |
| `terraform state list` | List state resources |
| `terraform state show <res>` | Inspect a resource |

## 7 COMMON FIX STRATEGIES

**CHECK PATHS**
Verify module source paths, include paths, and working directory.

**VERIFY INPUTS**
Ensure all required variables are provided.

**FIX DEPENDENCIES**
Apply dependencies first or use correct dependency outputs.

**CHECK STATE**
Ensure remote state exists and backend is accessible.

**REFRESH AUTH**
Update credentials or assume the correct role.

**CHECK PROVIDER**
Confirm region, permissions, and provider version.

### JUNIOR ENGINEER TIP

Take your time and follow the flow. Most failures are small issues in configuration, inputs, or permissions. Read, understand, and fix the root cause, not the symptom.

**REMEMBER**
Don't ignore warnings.
They are clues to future failures.

# PRODUCTION TERRAGRUNT ARCHITECTURE

## DESIGNING INFRASTRUCTURE THAT IS SCALABLE, SECURE, AND MAINTAINABLE

### 1 REUSABLE TERRAFORM MODULES

- Encapsulate best practices
- Reusable across environments
- Versioned and tested
- Small, focused, composable

MODULES ARE THE BUILDING BLOCKS.

### 2 LIVE INFRASTRUCTURE CONFIG

- Terragrunt manages how modules are deployed
- Environment, account, region, and inputs
- DRY with inheritance
- Easy to scale and navigate

CONFIG IS THE ORCHESTRATION LAYER.

### 3 MULTIPLE CLOUD ACCOUNTS

- Separate accounts by environment or team
- Strong isolation
- Least privilege access
- Centralized governance

ISOLATE RISK BY ACCOUNT.

### 4 MULTIPLE REGIONS

- Deploy across regions for resilience
- Region-aware configs
- Data residency
- Failover and DR

DESIGN FOR GLOBAL RESILIENCE.

### 5 DEV / STAGING / PRODUCTION ISOLATION

DEV → Rapid Iteration

STAGING → Validation & Testing

PRODUCTION → Stability & Reliability

STRICT ISOLATION PREVENTS ACCIDENTS AND DRIFT.

### 6 REMOTE STATE

- Store state in remote backend (e.g., S3 + DynamoDB)
- State locking prevents concurrent writes
- Encrypt and restrict access
- Separate state per environment

STATE IS CRITICAL. PROTECT IT.

### 7 MODULE VERSION PINNING

- Pin module versions (e.g., ref = "v1.2.3")
- Avoid unintentional breaking changes
- Upgrade intentionally

PIN TODAY, STAY STABLE TOMORROW.

### 8 CI/CD EXECUTION

COMMIT → PLAN → REVIEW & APPROVAL → APPLY

- Automated plan on PR
- Manual approval gate
- Apply only after approval
- Audit every change

AUTOMATE THE PROCESS, NOT THE RISK.

### 9 DEPENDENCY MANAGEMENT

- Use dependency blocks
- Manage order automatically
- Pass outputs safely
- Avoid circular dependencies

DEPENDENCIES CREATE ORDER AND RELIABILITY.

### 10 SECURITY BOUNDARIES

- Least privilege IAM
- Separate roles per environment
- No long-lived credentials
- Secrets in secure stores
- Restrict network access

SECURITY BY DESIGN, NOT BY ACCIDENT.

### 11 REPOSITORY GOVERNANCE

- Clear repo structure
- Code owners & reviews
- Standards & conventions
- Branch protection
- Audit and compliance

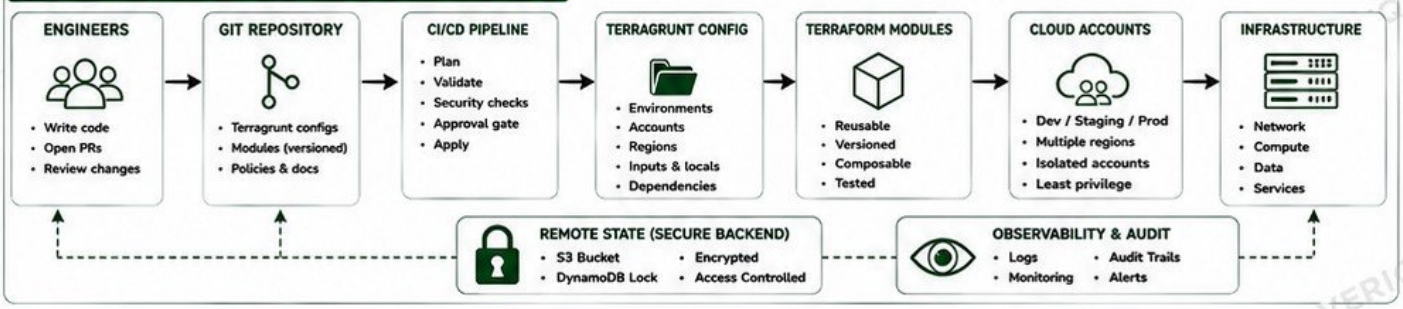
GOVERNANCE KEEPS TEAMS ALIGNED.

### 12 AVOID GIANT ROOT CONFIGURATIONS

- Keep root configs thin
- Use includes and locals
- Split by domain or stack
- Avoid huge monolith configs

SMALL, FOCUSED CONFIGS SCALE BETTER.

### 13 OVERALL PRODUCTION TERRAGRUNT ARCHITECTURE




### JUNIOR ENGINEER TIP

A good Terragrunt architecture is simple, modular, secure, and well-organized. Design for the future, not just for today.

### CHECKLIST FOR PRODUCTION READINESS

- ☑ Modules versioned and pinned
- ☑ Environments isolated
- ☑ Remote state secure and locked
- ☑ CI/CD with plan & approval
- ☑ Access least privilege
- ☑ Dependencies validated
- ☑ Docs and runbooks available
- ☑ Auditable and observable

### REMEMBER

Infrastructure is harder to rollback than applications. Design carefully. Change safely. Operate confidently.

# FINAL PRODUCTION LAB,

## BUILD A MULTI-ENVIRONMENT INFRASTRUCTURE STACK

YOU WILL BUILD, DEPLOY, BREAK, AND FIX A REAL INFRASTRUCTURE STACK USING TERRAGRUNT.
THE GOAL: PRACTICE A SAFE, REPEATABLE, PRODUCTION-WORTHY WORKFLOW.

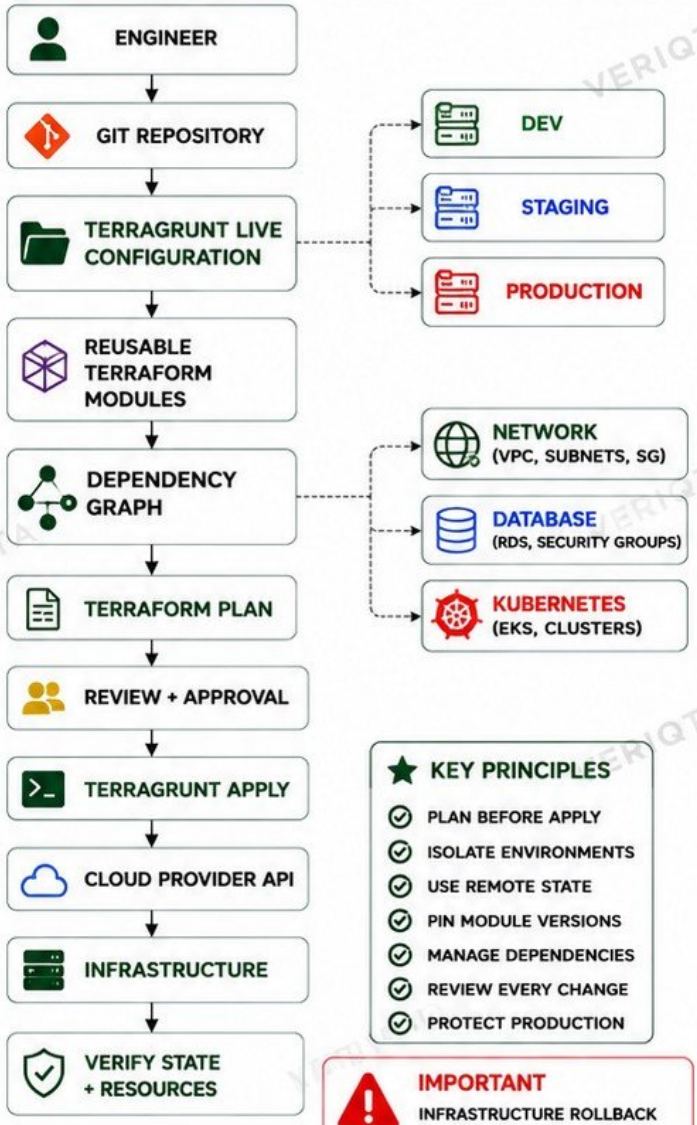
### LAB STEPS (FOLLOW THE ORDER)

1. CREATE REUSABLE TERRAFORM MODULES
2. BUILD THE TERRAGRUNT DIRECTORY STRUCTURE
3. CONFIGURE ROOT / SHARED SETTINGS
4. CREATE DEV AND PRODUCTION ENVIRONMENTS — DEV | STAGE | PROD
5. CONFIGURE REMOTE STATE
6. GENERATE PROVIDER CONFIGURATION
7. DEPLOY A VPC
8. DEPLOY DEPENDENT INFRASTRUCTURE (SUBNETS, IGW, ROUTE TABLES, ETC.)
9. PASS DEPENDENCY OUTPUTS
10. RUN INITIALIZATION AND VALIDATION
11. GENERATE AND REVIEW PLANS
12. APPLY INFRASTRUCTURE
13. INTRODUCE A DELIBERATE CONFIGURATION FAILURE
14. DIAGNOSE THE FAILURE
15. CORRECT THE CONFIGURATION
16. RE-PLAN BEFORE APPLYING
17. VERIFY INFRASTRUCTURE STATE
18. REVIEW PRODUCTION SAFETY

### SUCCESS CRITERIA

- Infrastructure deployed in DEV and PROD successfully
- Plans are clean and reviewed
- Failure was diagnosed and fixed
- Final state matches the expected architecture
- Production safety practices are followed

### FINAL ARCHITECTURE




### KEY PRINCIPLES

- PLAN BEFORE APPLY
- ISOLATE ENVIRONMENTS
- USE REMOTE STATE
- PIN MODULE VERSIONS
- MANAGE DEPENDENCIES
- REVIEW EVERY CHANGE
- PROTECT PRODUCTION

**IMPORTANT**

INFRASTRUCTURE ROLLBACK IS LIMITED AND NOT ALWAYS POSSIBLE. PREVENT, DETECT, AND CORRECT EARLY.

### LEARNING PROGRESSION

TERRAFORM PROBLEM → TERRAGRUNT FUNDAMENTALS → DRY CONFIGURATION → STATE → ENVIRONMENTS → DEPENDENCIES → MULTI-UNIT OPERATIONS → SECURITY → CI/CD → TROUBLESHOOTING → PRODUCTION